

LATE TRANSACTION HANDLING & HISTORICAL REVENUE CORRECTION

Data Engineering Project Report

Technologies Used Databricks | Apache Spark | PySpark | Delta Lake | Auto Loader | Spark SQL | Python

Submitted by: Aditya Kumar

Student Id: CT_CSI_DE_1086

1. Executive Summary

The Late Transaction Handling & Historical Revenue Correction project implements a scalable data engineering pipeline designed to address the problem of delayed transaction ingestion and inaccurate historical revenue reporting.

In real-world transactional systems, a transaction may occur on one date but enter the data platform several days later. If revenue reports are generated before these delayed transactions arrive, historical revenue figures become incomplete and require retroactive correction.

To address this problem, the project implements a Medallion Architecture (Bronze → Silver → Gold) using Databricks, PySpark, and Delta Lake.

The Bronze layer ingests raw CSV transaction data using Databricks Auto Loader and stores it as Delta data. The Silver layer performs type standardization, validation, deduplication, and data-quality processing. The Gold layer produces daily revenue reports based on the transactions available at the initial reporting point.

A dedicated late-transaction detection process identifies records where:

`ingestion_date > txn_date`

The affected historical transaction dates are then isolated. Instead of rebuilding the entire Gold dataset, the pipeline recalculates revenue only for those affected dates using all valid Silver transactions and updates the Gold Delta table through a Delta Lake MERGE operation.

The final implementation also includes watermark control, data-quality validation, auditability through Delta history/time travel, automated testing, and an end-to-end demonstration notebook.

The supplied dataset contains 2,000 transactions, of which 1,415 transactions (70.75%) are late-arriving. The pipeline successfully validates the complete correction workflow and produces a final corrected revenue of \$967,793.88 in the implemented validation environment.

2. Problem Statement

2.1 Business Context

In retail, e-commerce, payment processing, and other transactional environments, data ingestion is rarely instantaneous.

Transactions may be delayed because of:

- Network connectivity issues
- Offline point-of-sale systems
- Third-party payment processors
- Batch processing schedules
- Delayed source-system synchronization
- Data pipeline delays

As a result, a transaction that occurred on one day may not reach the reporting system until several days later.

For example:

Transaction Date : 2024-01-05

Ingestion Date : 2024-01-12

The transaction belongs to the revenue of **January 5**, even though it was ingested on **January 12**.

If the January 5 revenue report has already been generated, the historical figure becomes understated unless the system performs a correction.

2.2 Core Challenges

Delayed Data Ingestion

Transactions can arrive after their actual transaction date.

Stale Historical Reports

Historical revenue reports may be generated before all transactions belonging to that date have arrived.

Expensive Full Reprocessing

A naive solution would recalculate the entire historical revenue table whenever a late transaction arrives. This creates unnecessary processing and increases computational cost.

Data Quality

Duplicate transactions, invalid transaction IDs, invalid dates, and invalid transaction amounts can affect revenue calculations.

Data Trust and Auditability

Historical revenue corrections must be traceable and should not compromise the integrity of existing data.

2.3 Business Impact

Impact Area	Problem	Business Consequence
Revenue Reporting	Historical revenue is incomplete	Incorrect financial reporting
Decision Making	Decisions use stale revenue	Poor forecasting and planning
Data Trust	Reports may not match actual transactions	Reduced stakeholder confidence
Processing Cost	Entire history may be recomputed	Higher compute consumption
Auditability	Historical corrections may be difficult to trace	Reduced transparency

3. Project Objectives

The primary objectives of the project are:

1. Build a complete Bronze-Silver-Gold data pipeline.
 2. Ingest raw transaction files using Databricks Auto Loader.
 3. Store pipeline layers using Delta Lake.
 4. Clean and validate transaction data in the Silver layer.
 5. Deduplicate transactions using transaction ID.
 6. Generate daily revenue reports in the Gold layer.
 7. Detect late-arriving transactions using transaction and ingestion dates.
 8. Identify only the historical dates affected by late transactions.
 9. Recalculate revenue only for affected historical dates.
 10. Correct Gold revenue using Delta Lake MERGE.
 11. Preserve unaffected historical records.
 12. Maintain incremental processing state using a watermark.
 13. Provide data-quality validation and quarantine handling.
 14. Provide auditability using Delta Lake history/time travel.
 15. Validate the pipeline through automated unit and pipeline-level tests.
-

4. Dataset

4.1 Dataset Description

The project uses the supplied:

Project_Dataset.csv

The dataset contains transactional information required to demonstrate delayed ingestion and historical revenue correction.

Dataset Schema

Column	Data Type	Description
txn_id	BIGINT	Unique transaction identifier
user_id	BIGINT	Customer/user identifier
txn_date	DATE	Actual date on which the transaction occurred
amount	DECIMAL(18,2)	Transaction revenue amount
ingestion_date	DATE	Date on which the transaction was ingested

4.2 Dataset Statistics

The supplied dataset contains:

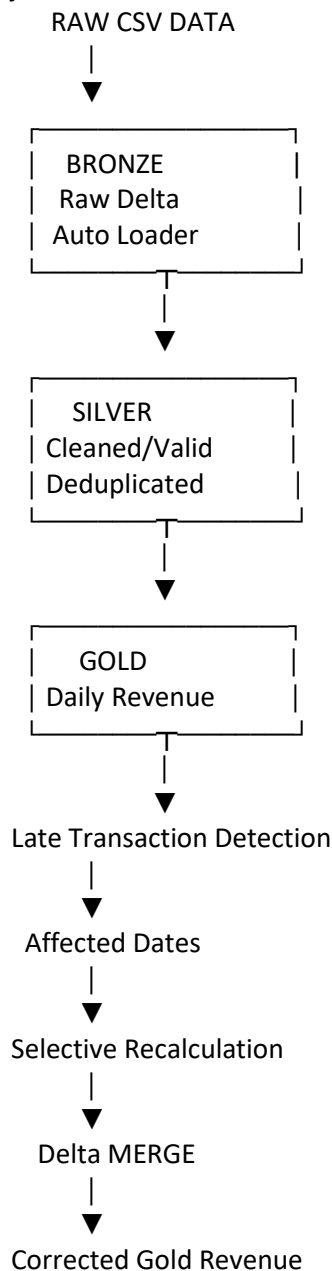
Metric	Value
Total Transactions	2,000
Unique Users	400
Transaction Dates	60
Transaction Date Range	2024-01-01 to 2024-02-29
Maximum Ingestion Date	2024-03-14
On-Time Transactions	585
Late Transactions	1,415
Late Transaction Percentage	70.75%
Affected Historical Dates	60
Null Values	0
Duplicate Transaction IDs	0
Negative Amounts	0
Zero Amounts	0

The high proportion of late-arriving records makes the dataset particularly suitable for demonstrating the project's historical correction mechanism.

5. Solution Architecture

5.1 Medallion Architecture

The project follows the **Medallion Architecture**, separating data processing into three logical layers:



5.2 Layer Responsibilities

Layer	Storage	Responsibility
Bronze	Delta Lake	Raw transaction ingestion
Silver	Delta Lake	Cleansing, validation, typing and deduplication
Gold	Delta Lake	Daily revenue reporting and historical correction

The architecture provides separation of concerns while allowing incremental processing and data-quality enforcement at each stage.

6. Technology Stack

Component	Technology	Purpose
Data Platform	Databricks	Pipeline execution environment
Compute	Apache Spark	Distributed data processing
Processing	PySpark	Transformations and business logic
Storage	Delta Lake	ACID tables, MERGE and history
Ingestion	Auto Loader	Incremental CSV ingestion
Query	Spark SQL	Analytical and validation queries
Language	Python	Pipeline implementation
Input	CSV	Transaction source
Output	Delta Tables	Bronze, Silver and Gold datasets

7. Implementation Details

7.1 Bronze Layer — Raw Ingestion

The Bronze layer is responsible for ingesting the raw transaction data without applying business-level cleansing.

Databricks Auto Loader is implemented using the cloudFiles format to detect and incrementally ingest CSV files.

The ingestion process uses:

- Streaming ingestion
- Schema handling
- Checkpointing
- File-level processing
- Delta Lake storage
- Source lineage metadata

The logical Bronze dataset is:

bronze_transactions

The Bronze layer preserves the raw structure of incoming transactions and acts as the first persistent layer of the pipeline.

7.2 Silver Layer — Cleansing and Validation

The Silver layer transforms Bronze data into a clean and reliable transaction dataset.

The following operations are performed.

Type Standardization

The pipeline converts:

txn_id → BIGINT

user_id → BIGINT

txn_date → DATE

ingestion_date → DATE

amount → DECIMAL(18,2)

Transaction ID Validation

Records with null transaction IDs are considered invalid and are prevented from entering the valid Silver dataset.

Deduplication

Duplicate transactions are identified using:

txn_id

Deterministic deduplication is implemented using window functions and row numbering.

Amount Validation

Negative and invalid transaction amounts are identified and excluded from valid revenue calculations.

Date Validation

Transaction and ingestion dates are validated before records enter the trusted Silver dataset.

Quarantine

Invalid records can be routed to a dedicated quarantine Delta table for inspection rather than silently being discarded.

The resulting trusted dataset is:

silver_transactions

8. Gold Layer — Daily Revenue

The Gold layer represents the business-facing revenue dataset.

Daily revenue is calculated by grouping valid transactions by transaction date:

daily_revenue = SUM(amount)

The primary business key is:

txn_date

The logical Gold table is:

gold_daily_revenue

8.1 Initial Revenue Snapshot

The project distinguishes between the initial reporting snapshot and the final corrected Gold state.

The initial reporting snapshot represents transactions available at the original reporting point.

For the implemented demonstration, the initial on-time population is used:

ingestion_date = txn_date

The resulting initial revenue is:

\$294,592.50

across:

585 on-time transactions

This establishes the incomplete historical state that the correction process subsequently addresses.

9. Late Transaction Detection

Late-arriving transactions represent the central business problem addressed by the project.

A transaction is classified as late when:

`ingestion_date > txn_date`

For example:

`txn_date = 2024-01-08`

`ingestion_date = 2024-01-15`

This transaction occurred on January 8 but was received on January 15 and therefore affects the historical revenue for January 8.

9.1 Dataset Results

The implemented pipeline identified:

Category	Count	Percentage
Total Transactions	2,000	100%
On-Time Transactions	585	29.25%
Late Transactions	1,415	70.75%

The result demonstrates that late-arriving data is a significant component of the supplied dataset.

10. Affected Historical Dates

Once late transactions are detected, the pipeline extracts their distinct `txn_date` values.

These dates represent historical reporting dates potentially affected by late-arriving transactions.

The process is:

Late Transactions



Select `txn_date`



DISTINCT



Affected Historical Dates

The dataset produced:

60 affected historical dates

The pipeline therefore avoids blindly recalculating every historical date.

11. Selective Historical Revenue Recalculation

For each affected historical date, the pipeline recalculates revenue using **all valid Silver transactions associated with that date**.

This includes:

- Transactions that arrived on time
- Transactions that arrived late

The calculation is conceptually:

Affected Dates



Filter Silver Transactions



Group by txn_date



SUM(amount)



Corrected Daily Revenue

This is critical because calculating revenue using only the late transactions would not produce the true historical total.

The corrected revenue must represent the complete set of valid transactions for the affected date.

12. Delta Lake MERGE

After recalculating affected historical dates, the corrected values are written into the Gold Delta table using a Delta Lake MERGE.

The merge key is: txn_date

The operation follows: WHEN MATCHED → Update daily_revenue

WHEN NOT MATCHED → Insert new historical date

This allows the pipeline to update only the records that actually require correction.

Unlike a full-table overwrite, the MERGE operation preserves unaffected Gold records.

13. Surgical Historical Correction

The key engineering principle of the project is **selective correction**.

A conventional approach might perform:

All Historical Transactions



Recalculate Entire Gold Table



Overwrite Gold

The implemented approach instead performs:

Detect Late Transactions



Identify Affected Dates



Recalculate Only Affected Dates



Delta MERGE



Corrected Gold

This reduces unnecessary processing and prevents unaffected historical records from being rewritten.

14. Watermark Control

The project includes a dedicated Delta-based watermark control mechanism.

The control structure contains:

Column	Description
table_name	Pipeline/table being tracked
last_processed_date	Latest successfully processed date

The watermark lifecycle is:

Read Current Watermark



Process Required Data



Complete Downstream Operations



Successful Commit



Advance Watermark

The watermark is updated only after successful downstream processing.

This supports incremental execution and prevents already processed data from being unnecessarily reprocessed.

15. Data Quality Checks

The pipeline implements multiple data-quality validations.

Check	Validation	Action
Null Transaction ID	txn_id IS NULL	Quarantine / exclude
Duplicate Transaction ID	Duplicate txn_id values	Deterministic deduplication
Negative Amount	amount < 0	Exclude / flag
Zero Amount	amount = 0	Validate according to business rules
Invalid Transaction Date	Invalid/null date	Reject / quarantine
Invalid Ingestion Date	Invalid/null date	Reject / quarantine
Amount Type	Numeric conversion	Cast / reject invalid values

For the supplied dataset, the validation results were:

Null transactions : 0

Duplicate transactions : 0

Negative amounts : 0

Zero amounts : 0

Invalid/inverted dates : 0

Therefore, all 2,000 supplied records passed the implemented baseline quality checks.

16. Auditability and Delta Time Travel

Delta Lake provides transaction history and time-travel capabilities that support auditability.

The project includes audit functionality for examining:

- Delta transaction history
- Table versions
- Before/after states
- Historical changes caused by correction operations

The intended correction lifecycle is:

Gold Before Correction



Late Transactions Detected



Historical Recalculation



Delta MERGE



Gold After Correction

Delta history can be inspected using DESCRIBE HISTORY, while previous table versions can be queried using Delta time-travel functionality.

This makes historical corrections traceable and provides evidence of changes made to reporting data.

17. End-to-End Pipeline Workflow

The complete pipeline follows these steps:

Step 1 — Raw Data Ingestion

CSV transaction files are provided as the source dataset.

Step 2 — Bronze Ingestion

Auto Loader detects and ingests incoming CSV data into the Bronze Delta layer.

Step 3 — Silver Transformation

Bronze records are cleaned, typed, validated, deduplicated, and quarantined where necessary.

Step 4 — Initial Gold Reporting

The initial reporting snapshot is generated using the transactions available at the reporting point.

Step 5 — Late Transaction Detection

Transactions where:

`ingestion_date > txn_date`

are identified.

Step 6 — Affected Date Identification

Distinct transaction dates associated with late transactions are extracted.

Step 7 — Selective Recalculation

Only affected dates are recalculated using all valid Silver transactions.

Step 8 — Delta MERGE

Corrected revenue values are merged into the Gold Delta table.

Step 9 — Watermark Update

The processing watermark is advanced after successful completion.

Step 10 — Validation

The corrected Gold output is compared against Silver-derived revenue.

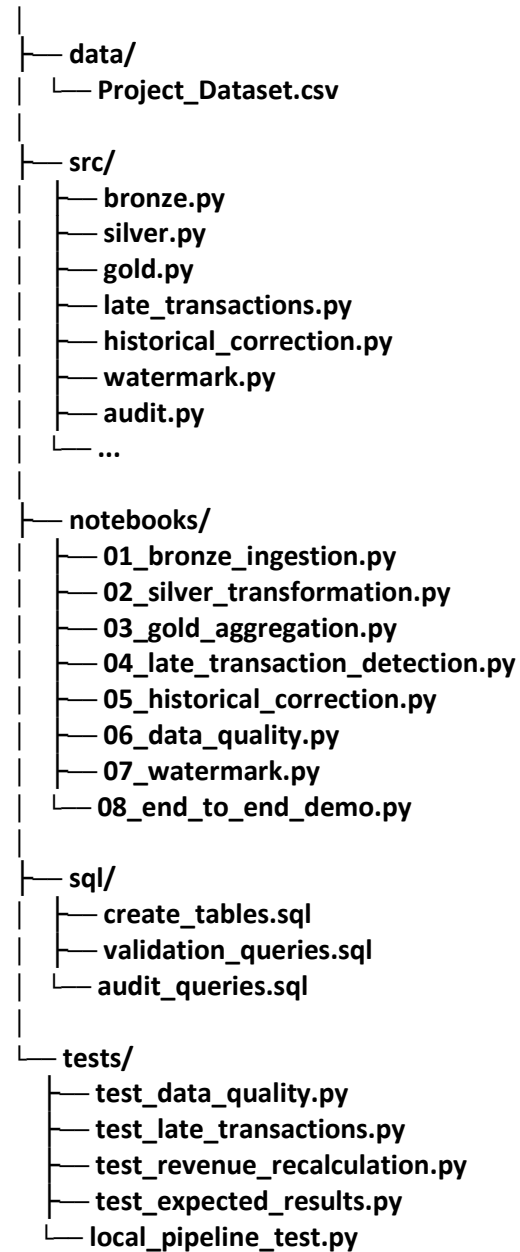
Step 11 — Audit

Delta history/time-travel capabilities can be used to inspect the correction.

18. Implementation Structure

The project is organized into modular source code, notebooks, SQL scripts, and tests.

Project



19. Testing and Validation

Testing was performed at two levels:

1. Unit testing
2. Pipeline-level validation

19.1 Unit Tests

The project contains **20 unit tests**.

Result:

20 / 20 PASSED

The tests cover:

- Data quality
- Late transaction detection
- Revenue recalculation
- Expected dataset results
- Auto Loader configuration
- Initial Gold snapshot behavior
- MERGE source restriction
- Watermark failure resilience
- Dataset date validation

19.2 Pipeline-Level Tests

The project contains **10 pipeline-level validation tests**.

Result:

10 / 10 PASSED

Test	Result
Dataset Loading	PASS
Type Conversion	PASS
Data Quality	PASS
Late Transaction Detection	PASS
Affected Date Identification	PASS
Revenue Recalculation	PASS
Delta MERGE Logic	PASS
Unaffected Date Preservation	PASS
Idempotency	PASS
End-to-End Revenue Invariant	PASS

20. Final Validation Results

The implemented pipeline produced the following validation results:

Metric	Result
Total Transactions	2,000
On-Time Transactions	585
Late Transactions	1,415
Affected Historical Dates	60
Initial Revenue	\$294,592.50
Late Revenue	\$673,201.38
Final Corrected Revenue	\$967,793.88

The final end-to-end validation established mathematical parity between the corrected Gold revenue and the corresponding valid Silver revenue:

Silver Revenue = \$967,793.88

Gold Revenue = \$967,793.88

This confirms the core revenue-correction invariant within the implemented validation pipeline.

21. Key Engineering Benefits

21.1 Surgical Historical Corrections

Only historical dates affected by late transactions are recalculated.

21.2 Reduced Processing

The solution avoids unnecessary full-table historical rewrites.

21.3 Data Quality

The Silver layer ensures that invalid and duplicate transactions do not directly affect revenue reporting.

21.4 ACID Transactions

Delta Lake provides transactional guarantees for updates to the Gold dataset.

21.5 Incremental Processing

Auto Loader and watermark control support incremental pipeline execution.

21.6 Auditability

Delta history and time-travel capabilities provide visibility into historical changes.

21.7 Scalability

PySpark and Databricks provide distributed processing capabilities suitable for significantly larger transaction volumes than the supplied dataset.

21.8 Reliability

Auto Loader checkpointing provides reliable incremental file ingestion and protects against duplicate processing during streaming execution.

22. Limitations

The project is designed as a data engineering pipeline and therefore does not include a custom web or frontend application.

The primary execution and demonstration environment is Databricks using PySpark, Spark SQL, Delta Lake, and Databricks notebooks.

The supplied dataset is relatively small compared with enterprise-scale transactional datasets.

Therefore, large-scale performance benchmarking was outside the scope of this implementation.

The project focuses specifically on delayed transaction ingestion and historical revenue correction rather than broader financial reporting, forecasting, or machine-learning functionality.

23. Future Scope

The solution can be extended in several directions:

1. Integration with additional transaction sources.
 2. Automated Databricks Workflows scheduling.
 3. Integration with BI tools such as Power BI or Tableau.
 4. Real-time operational monitoring.
 5. Advanced data-quality monitoring and alerting.
 6. Automated anomaly detection for unusual transaction delays.
 7. Additional business dimensions such as region, product, store, or transaction type.
 8. More sophisticated incremental processing based on event-time and ingestion-time windows.
 9. Enterprise-grade observability and alerting.
 10. CI/CD integration for automated deployment and testing.
-

Conclusion

The Late Transaction Handling & Historical Revenue Correction project demonstrates a practical data engineering solution to the problem of delayed transaction ingestion and stale historical revenue reporting.

By combining Databricks, PySpark, Delta Lake, Auto Loader, and Medallion Architecture, the solution separates raw ingestion, data cleansing, and business-level aggregation into well-defined processing layers.

The key innovation is the selective historical correction mechanism. Instead of rebuilding the entire Gold dataset when late transactions arrive, the pipeline identifies the specific historical dates affected by those transactions, recalculates revenue for only those dates using all valid transaction data, and updates the Gold Delta table through a targeted MERGE operation.

The implementation further incorporates data-quality validation, deterministic deduplication, quarantine handling, watermark control, automated testing, and Delta Lake auditability.

For the supplied dataset, the pipeline successfully processes 2,000 transactions, identifies 1,415 late-arriving transactions, isolates 60 affected historical dates, and produces a final corrected revenue of \$967,793.88 in the implemented validation environment.

The project therefore demonstrates the core engineering principle of incremental, reliable, and auditable historical correction without unnecessary full-table recomputation.
